

Final Project Report: Clustering and Initialization

Federico Maspero

Department of Computer Science, University of Milan

Nicolò Cesa-Bianchi

Luigi Foscarì

Emmanuel Esposito

June 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

This report describes the results obtained in the final project for the Statistical Methods for Machine Learning exam. The project, chosen from a list provided by Professor Cesa-Bianchi, was focused on studying the impact of initialization on clustering performance.

Clustering is an unsupervised learning task in which a dataset is partitioned into groups, called clusters, so that points within the same group are similar to one another and share less similarities with points in other groups.

To study the impact of initialization, the clustering algorithms `k-means` and `k-means++` have been implemented from scratch, and tested on the same datasets to measure differences in performance.

All of the code developed for this project can be viewed and tested in the Jupyter notebook available in the project's GitHub public repository [2].

1.1 Theoretical background of K-means clustering

As previously stated, k-means is an unsupervised learning algorithm that performs a clustering task: when given a set of data points $\{x_1, \dots, x_n\} \subset R^d$, the goal is to divide them into k clusters based on the similarity of those points. Each cluster is represented by a centroid, computed as the mean of all points assigned to it.

The most commonly used metric to evaluate how well the task was performed is referred to as inertia:

$$\phi = \sum_{i=1}^n \min_{j \in \{1, \dots, k\}} \|x_i - c_j\|^2$$

Inertia is the sum, over all points, of the squared distance from each point to its nearest centroid.. The lower the inertia, the more the clusters are well separated and defined.

To find those final centroids and define the clusters, k-means operates in the following way:

1. randomly choose k points among the given data points in dataset X , and assign them as centroids $\{c_1, \dots, c_k\}$
2. for each centroid c_i set the cluster C_i to be the subset of points in X that are closer to c_i than to c_j for all $j \neq i$
3. for each $i \in \{1, \dots, k\}$ set c_i to be the center of mass of all points in C_i : $c_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$
4. repeat steps 2 and 3 until convergence, when the set of centroids doesn't change between iterations

The initialization of k-means is random, having k points chosen arbitrarily through set X and assigned as centroids. Because the k-means objective is non-convex, the final result depends heavily on this starting choice, where different initial centroids can lead the algorithm to different solutions. This leads k-means to only be able to reach a local optimum, which often lies far from the globally optimal solution.

This is the reason that brought Arthur and Vassilvitskii [1] to develop a variant of the original algorithm, k-means++ which, instead of a random draw, applies a probabilistic initialization strategy to the selection of the first set of centroids. This version only chooses the first point at random, and chooses the others with a probability calculated in proportion to their distance to the nearest already selected point.

The steps taken by the `k-means++` algorithm are the following

1. choose an initial random centroid c_1 from X
2. choose the next centroid c_i selecting a different point $x \in X$ with probability $p(x') = \frac{D(x')^2}{\sum_{x \in X} D(x)^2}$, where $D(x)$ is the shortest distance from x to its closest centroid
3. repeat step 2 until the desired k centroids are selected
4. proceed as regular `k-means` would

The idea behind this procedure is to ensure that the starting clusters are well spread out through the dataset. This makes it less likely to generate bad starting configurations, which in turn should lead to more consistent final clusterings.

Arthur and Vassilvitskii proved that the expected inertia of the solution produced by `k-means++` satisfies the following bound:

$$\mathbb{E}[\phi] \leq 8(\ln k + 2) \phi_{\text{OPT}}$$

where ϕ_{OPT} is the inertia of the optimal clustering. This result guarantees that the solution found by `k-means++` should never be too far from the optimum.

2. Datasets

This section will showcase the datasets selected for the project. As per the assignment, the calculations were carried out both on a synthetic dataset, and a real-world high-dimensional one

The synthetic dataset was generated by sampling 500 points from a Gaussian distribution, the result of which can be seen in figure 1. The four cluster centers were manually chosen at $[(0, 0), (10, 10), (0, 10), (10, 0)]$, forming a square arrangement that ensures good separation between clusters. The number of points assigned to each cluster was randomly drawn, resulting in sizes of 120, 128, 114 and 138 respectively. This imbalance was introduced deliberately to better resemble a realistic dataset scenario.

A fixed random seed was used for data generation, ensuring that subsequent executions of the notebook will always produce the same starting dataset.

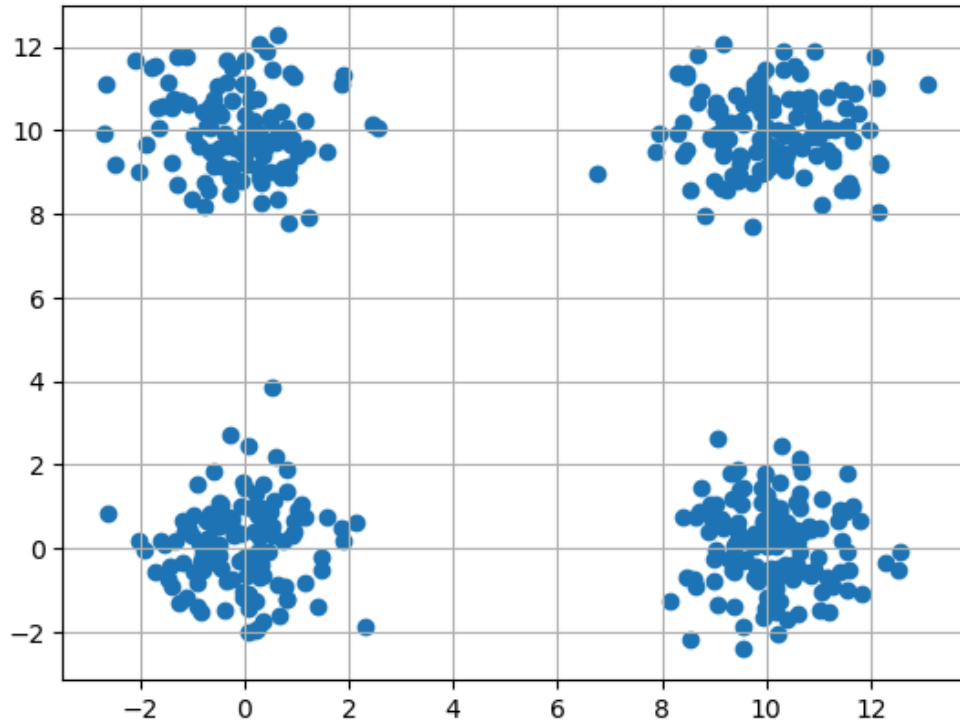


Figure 1: Synthetic dataset distribution

In terms of real-life high-dimensional dataset, the chosen one for this project was MNIST, like the Professor suggested in the project's assignment. MNIST is a dataset containing 70,000 grey-scale images of hand-written digits, where every 28×28 pixel image is converted into a 784-dimensional vector representing the grey values of each pixel.

The dataset was loaded from the scikit-learn library. Given its extremely large size, a random subset of 2000 elements was selected and used, so as to ease the computation times. The data is also normalized, by converting the pixel values into `float64` and dividing by 255, bringing each element in a $[0, 1]$ range.

Just like for the synthetic dataset, a random seed was also used for the selection of the 2000 elements from MNIST, so to ensure that subsequent executions of the notebook will always produce the same starting dataset.

3. Parameters and metrics of evaluation

This section describes the parameters of the experiments performed: how the algorithms were setup, the number of trials, the recorded results and how those results were evaluated.

The objective of the experiment was to perform multiple runs of both `k-means` and `k-means++`, compare the final results of each and determine whether or not the difference in initialization had a significant impact on them.

Both algorithms were implemented in Python from scratch, only using `numpy` - `matplotlib` was used to plot datasets and results. The implemented algorithms follow the working flows described in section 1.1.

The parameters of the experiments were set as follows:

Hyperparameter	Synthetic dataset	MNIST
<code>n_clusters</code>	4	10
<code>n_runs</code>	100	50
<code>max_iter</code>	100	100
<code>tolerance</code>	<code>np.allclose</code> defaults (<code>atol=1e-8</code>)	<code>np.allclose</code> defaults (<code>atol=1e-8</code>)

Table 1: Experiment hyperparameters

The number of clusters differs between the two experiments because of the nature of the datasets: the synthetic one was clearly designed around 4 clusters of points, while MNIST is a collection of images of digits, which has 10 different possible clusters of similar items.

Note that another difference is the number of runs performed: on the synthetic dataset is 100, while on MNIST is only 50. This was done, again, to ease computation times: given the much larger number of items and the extremely high dimensionality, each iteration has a much higher computational cost. A fixed random seed was applied to each individual run, using the run index as the seed value. This ensures that the exact results reported in this document are reproducible when the notebook is executed.

The results were evaluated by looking at the following values, calculated over the set number of runs:

- Average inertia
- Best inertia
- Worst inertia
- Standard deviation
- Average number of iterations

As previously stated, the main metric used for evaluating the results is inertia, by looking at the average, best and worst cases. The standard deviation of the inertia is an indicator of the variability of the results, and it allows to have an idea of how stable the algorithms are. Finally, the average number of iterations has also been included, as an indicator of convergence speed: a lower value suggests the algorithm reached a stable solution more quickly, which can be an indirect indicator of initialization quality.

4. Results

This section will present the experimental results obtained over the multiple runs of `k-means` and `k-means++`. The evaluation will be based on a comparison of the parameters stated in section 3

4.1 Synthetic dataset

The first experiment was conducted on the synthetic dataset. The results, reported in table 2 and also available on the Jupyter notebook, show a significant improvement in performance of `k-means++` over `k-means`.

Metric	K-Means	K-Means++
Average inertia	3441.03	1199.82
Best inertia	949.70	949.70
Worst inertia	7599.09	7574.61
Std inertia	2998.16	1227.49
Average iterations	5.76	2.71

Table 2: Comparison of `k-means` and `k-means++` results on synthetic data

The most notable difference is obviously in the average inertia, which shows roughly a 65.15% improvement for `k-means++` over `k-means`. While both algorithms reach the same best case, being the optimal solution, the worst cases and standard deviation vary drastically, showing much inconsistency in the results of the regular `k-means`, which can converge on extremely bad solutions with poor local minima and very high inertia.

The best and worst case results for both algorithms are shown in figures 2 and 3

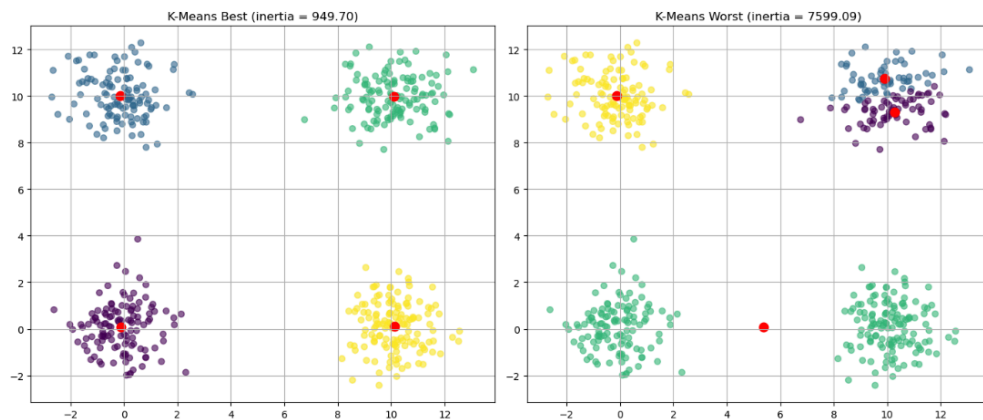


Figure 2: Best and worst case results for `k-means`

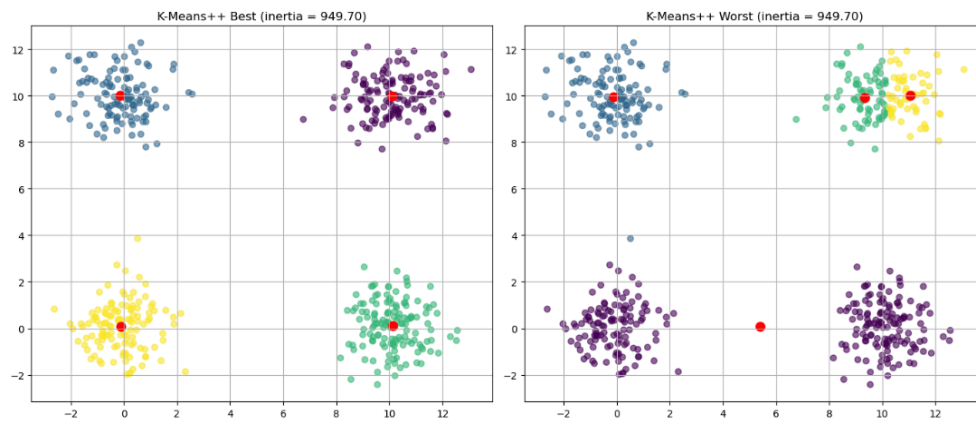


Figure 3: Best and worst case results for k -means++

The distribution of inertia values across all 100 runs, shown in Figures 4 and 5, makes this difference even clearer. Standard k -means produces a wide spread of results with several high-inertia outliers, while k -means++ concentrates its results in a much narrower and lower range.

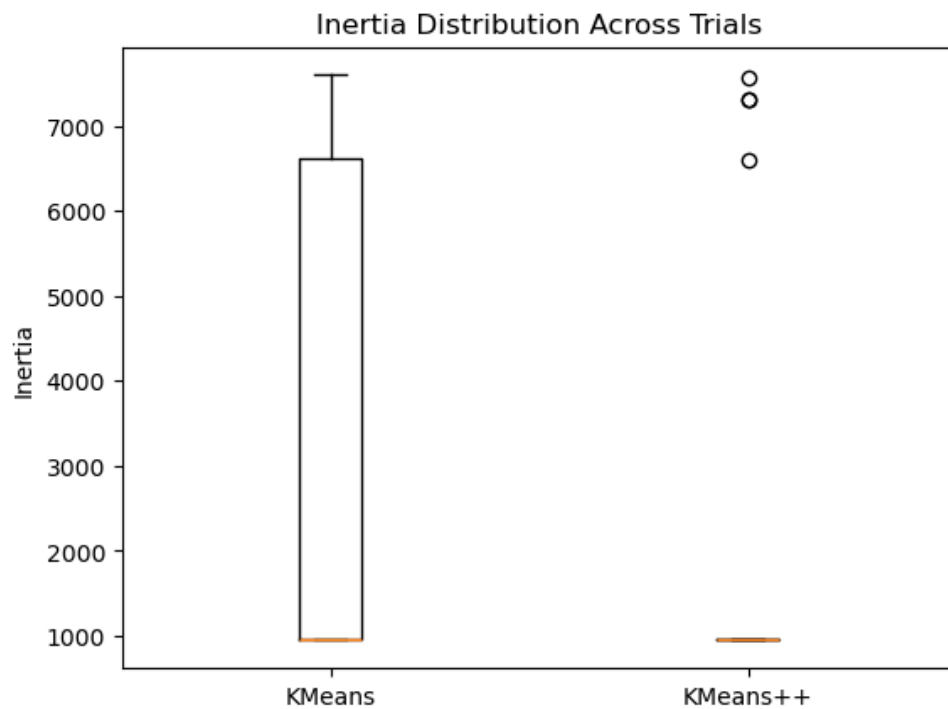


Figure 4: Boxplot of inertia results across 100 runs on synthetic dataset

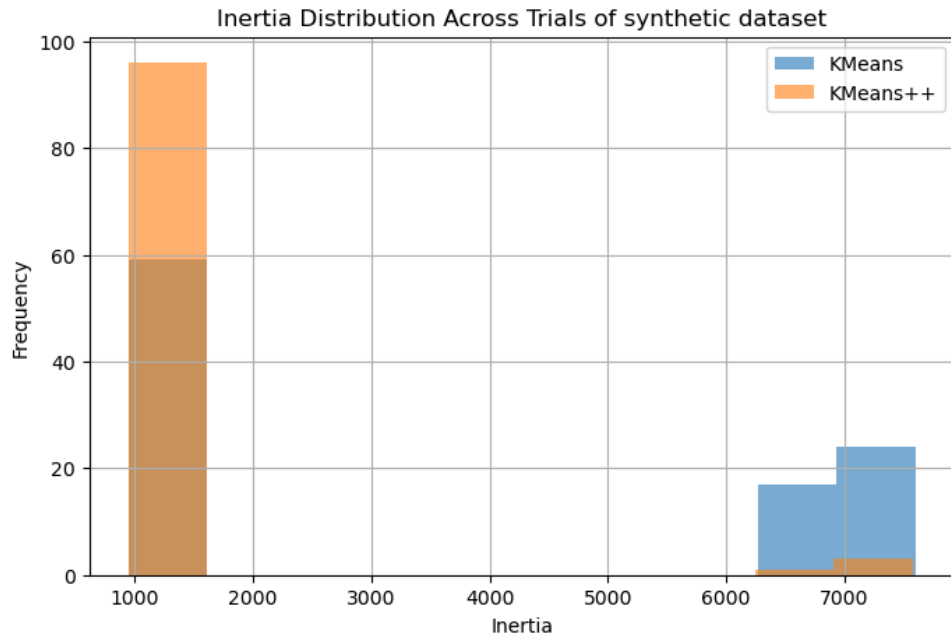


Figure 5: Bar graph of inertia distribution across 100 runs on synthetic dataset

Another interesting result to note is the average number of iterations required to reach convergence. Even on this relatively simple dataset, the smarter initialization allows the algorithm to reach a stable solution in roughly half the iterations, confirming that starting from better-spread centroids reduces the amount of adjustment needed during the optimization.

4.2 Real-life dataset: MNIST

The second experiment was conducted on the MNIST dataset. Given the higher complexity of the data, this proved to be a much more challenging test for both algorithms. The results are reported in table 3 and are also available on the Jupyter notebook.

Metric	K-Means	K-Means++
Average inertia	78461.27	78433.25
Best inertia	77983.11	77988.55
Worst inertia	79594.44	79025.34
Std inertia	313.94	265.77
Average iterations	29.68	29.72

Table 3: Comparison of k -means and k -means++ results on MNIST

Unlike for the synthetic dataset, the difference between the two algorithms in this case is negligible. The average inertia values, which are much higher than the synthetic case, are also nearly identical, barely a 0.036% difference

between the two. The distribution of inertia values across all 50 runs, shown in Figure 6, confirms this: both algorithms produce results concentrated in a very similar range. `k-means` actually out-performs `k-means++` in finding the optimal solution the most times, as shown by the bar graph in figure 7.

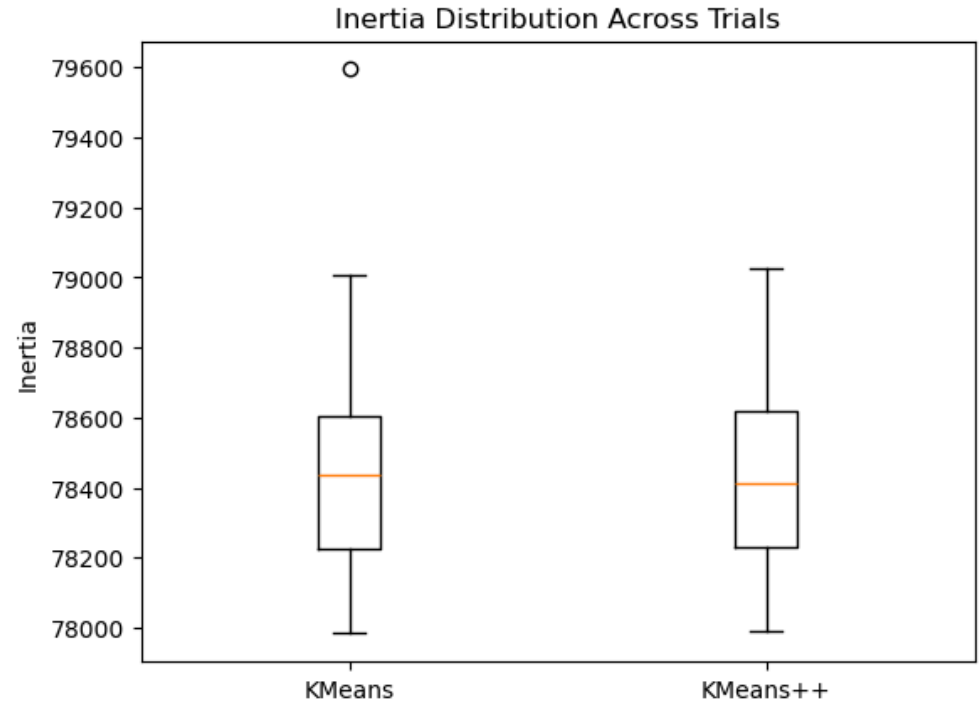


Figure 6: Box plot of inertia distribution across 50 runs on MNIST

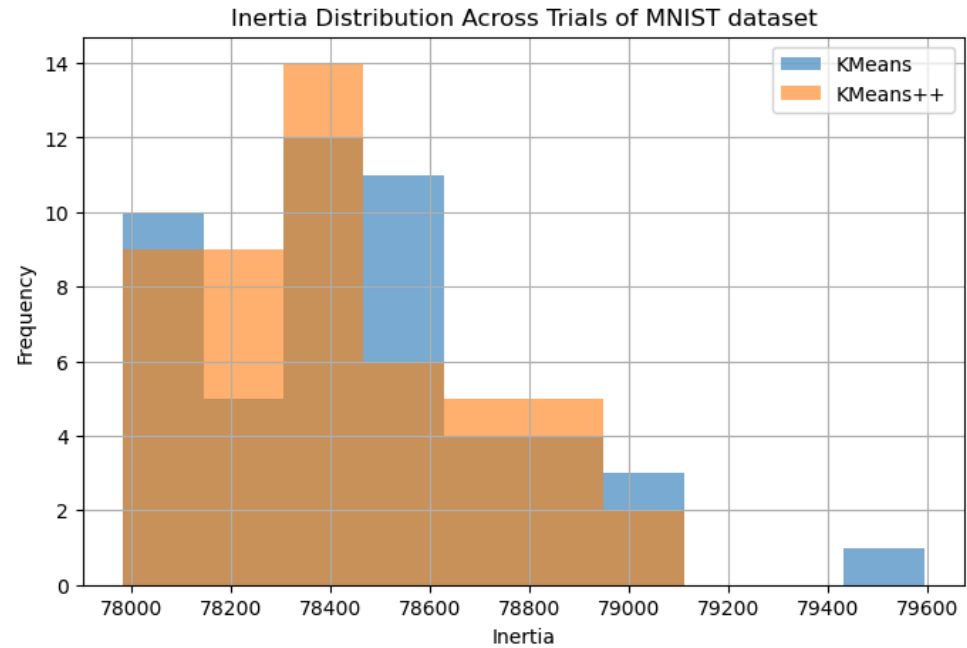


Figure 7: Bar graph of inertia results across 50 runs on MNIST

The standard deviations, while slightly lower for `k-means++`, are both similar enough to indicate that both algorithms converge to similarly stable solutions regardless of initialization strategy.

Notably, the average number of iterations is also virtually the same (29.68 vs 29.72), with `k-means++` showing no meaningful advantage in convergence

speed.

It is worth noting that the results reported here were obtained using a fixed random seed for reproducibility. When running the algorithms without a fixed seed, the results varied considerably across executions, sometimes showing `k-means++` performing slightly better, other times performing similarly or worse than regular `k-means`. This further confirms that, on high-dimensional data, the effect of initialization is inconsistent and dataset-dependent, unlike the clear and stable advantage observed on the synthetic dataset.

5. Extension: Overlapping clusters

One of the proposed extensions for this project was to test the two algorithms over a dataset with overlapping clusters. To do so, a synthetic dataset was created in a similar manner to the one used previously in section 4.1, with a few adjustments: the 4 centers were not placed on a square pattern, and the standard deviation was raised to $\sigma = 4$. This generated a much more spread out and chaotic distribution of points, in which the four clusters overlap, shown in the following figure 8.

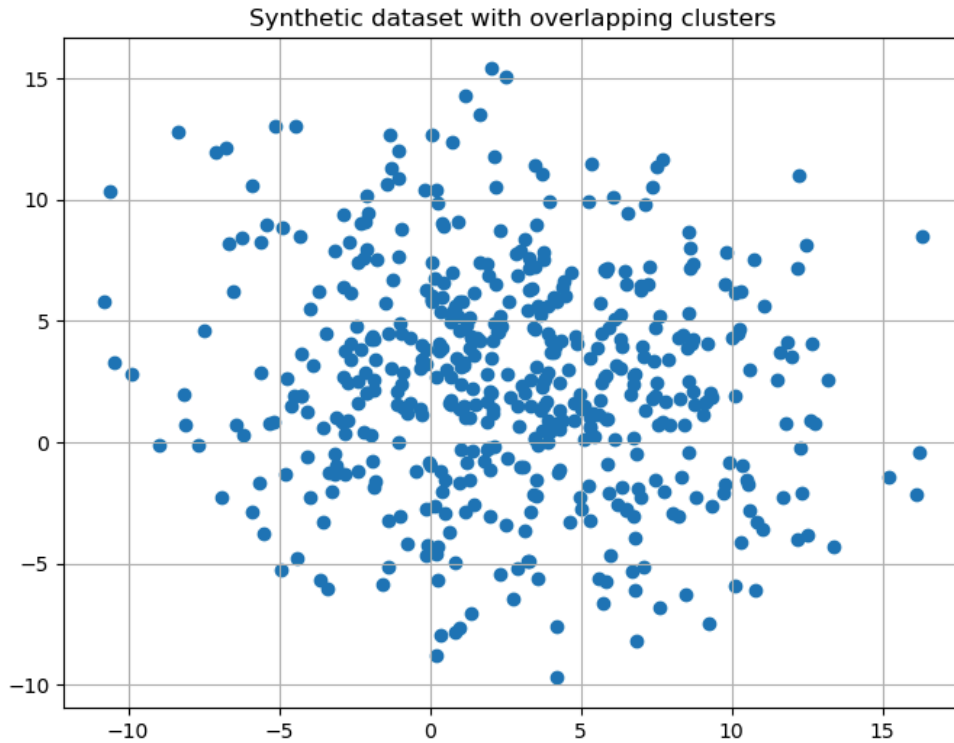


Figure 8: Distribution of points of the synthetic dataset with overlapping clusters

The parameters for the experiment were kept the same as the ones for the synthetic dataset experiment, as reported in table 4.

Hyperparameter	Overlapping dataset
n_clusters	4
n_runs	100
max_iter	100
tolerance	np.allclose defaults (atol=1e-8)

Table 4: Experiment hyperparameters for overlapping dataset

The results of the experiment can be viewed on the Jupyter notebook or on table 5:

Metric	K-Means	K-Means++
Average inertia	8067.87	8049.62
Best inertia	7938.47	7938.34
Worst inertia	9365.08	8661.45
Std inertia	188.92	116.48
Average iterations	16.44	17.51

Table 5: Comparison of *k*-means and *k*-means++ results on overlapping clusters

Figures 9 and 10 compare the best and worst run of each algorithm. This makes it rather clear to see that the difference between the two methods is much smaller than it was on the regular synthetic dataset: both the numbers in Table 5 and the plots show `k-means` and `k-means++` converging to roughly the same best and worst cases, with only the worst run separating them by any noticeable margin.

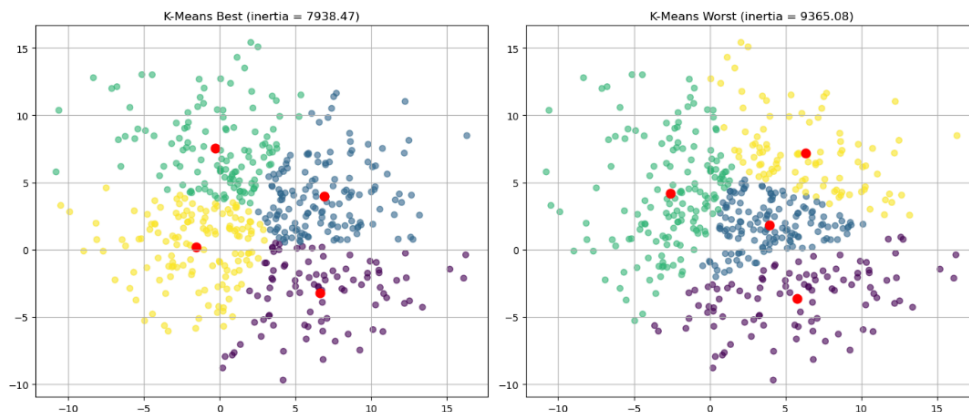


Figure 9: Best and worst case results for *k*-means over the overlapping clusters dataset

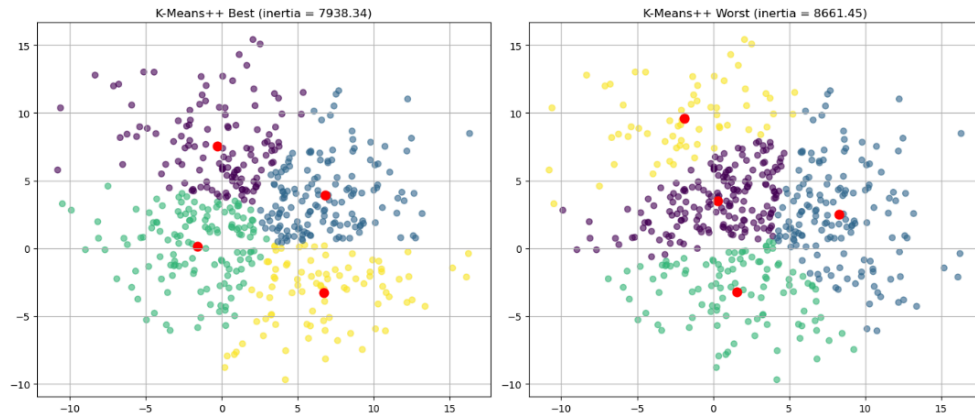


Figure 10: Best and worst case results for k -means++ over the overlapping clusters dataset

The difference in average inertia between the two algorithms is minimal, being only a 0.23% improvement, also visible in the graphs in figures 11 and 12. Both algorithms are capable of finding virtually the same best case solution, suggesting that when clusters overlap significantly, the initialization strategy has a limited impact on the average outcome.

`k-means++` shows better results for the worst case scenario and stability, with a notably lower standard deviation. This means that the algorithm is more capable in avoiding catastrophic failures, even when the average performance is overall similar.

An interesting result to note is the average number of iterations, showing that `k-means++` has actually been marginally out-performed by `k-means`. This was unexpected, as it never happened with the two previous datasets.



Figure 11: Boxplot of inertia distribution across 100 runs on overlapping dataset

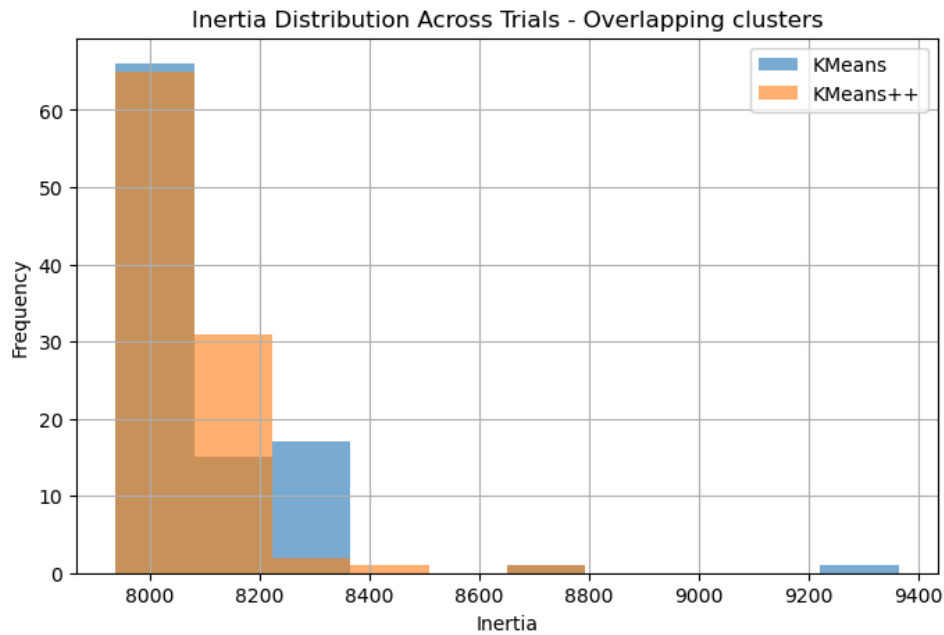


Figure 12: Bar graph of inertia results across 100 runs on overlapping dataset

6. Conclusion

In conclusion, this project has shown that initialization can play an important role in improving the performance of clustering algorithms, but its impact is heavily dependent on the structure of the dataset that is being analyzed.

The synthetic dataset proved to be a great example of the impact of initialization. `k-means++` achieved a clear and substantial improvement over standard `k-means`, reducing average inertia by approximately 65%, showing a much lower variability and substantially reducing the number of required iterations.

On the MNIST dataset, on the other hand, the effects of initialization were much more limited. The two algorithms produce nearly identical results on all metrics of evaluation. Also, testing the algorithms outside of the set random key yielded even more varied and less consistent results. This highlights how, with more complex data, initialization may not influence enough the final results to truly matter.

Finally, the dataset with overlapping clusters also didn't show much improvement between the two algorithms. Showing a nearly identical average inertia and both reaching the same best solution, the most notable result is the average number of iterations required to reach consistency, for which `k-means++` actually performs slightly worse than regular `k-means`. All these results indicate that the smarter initialization does not necessarily translate into faster convergence when cluster boundaries are ambiguous.

In conclusion, analyzing these results suggests that initialization holds value in clustering algorithms, but oftentimes it is outweighed by the characteristics of the data being processed. For low-dimensional data with distinct groups, `k-means++` performs much better than its regular counterpart, generating much more consistent results. For high-dimensional or heavily overlapping data, the choice of initialization seems to actually have a far lesser impact, in some cases going as far as performing marginally worse than random initialization. Still, the results stay consistent with the guarantee proven by Arthur and Vassilvitskii: `k-means++` managed to never fail catastrophically in any of the experiments.

Bibliography

- [1] David Arthur and Sergei Vassilvitskii. *k-means++: The advantages of careful seeding*. In Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 1027–1035, 2007.
- [2] Federico Maspero. *Project GitHub repository*.
https://github.com/Femaspe02/MLProject_MasperoFederico, 2026.